

Reference-Relative Protocol-Profile Comparison: A Reproducible, Source-Anchored Method for Situating Bitcoin’s Descendant Chains Against Historical References

Parth Mauria Saxena

Independent Researcher

`parthms.id@gmail.com`

14 September 2026

Abstract

“Which chain is closest to the original Bitcoin?” is usually answered as narrative or advocacy. We give a *reproducible* answer to a *narrower, well-posed* question: across a fixed set of consensus-protocol axes, how far do each descendant chain’s rules sit from a chosen historical reference? Every specified cell of the comparison is an **encoding with a stated warrant**: a consensus value decided by a stated criterion, at a frozen evaluation date, against a primary source (a BIP, an upgrade specification, or a self-describing commit) — or, for the 9 cells that assert the *absence* of a rule, against chronology, since non-adoption leaves no record to cite — with **29 of the 95 chain cells additionally verified by fetching their source mechanically** (Section 7). So the whole table is **machine-recomputable** from that encoding. **We are careful not to claim more.** Reproducibility means that anyone scoring the same encoding gets the same number; it does not make the encoding the uniquely correct reading of the protocol. **Judgement is not eliminated, it is relocated to individuation** — which axes exist, how finely they are cut, and what each state is called — where it is visible and can be perturbed on purpose, and we perturb it (Section 5). We report a **mismatch rate** and an explicit **coverage** for each reference→chain pair (the mismatch rate is undefined where coverage is zero), and four sensitivity analyses over axis choice and label granularity. Applied to BTC, BCH, BSV, XEC and BTG under three references (the 2008 whitepaper, the November 2008 pre-release, and the January 2009 v0.1.0 client), the method shows that the comparison is *degenerate* under the whitepaper (coverage $1/19 \approx 0.053$), *low-coverage* under the preview, and only well-posed under v0.1.0 — a pattern the axis rule partly builds in, since most axes are admitted because a rule changed after January 2009, and the

earlier references specify almost none of them — where, on the enumerated axes, BSV carries the fewest mismatches, a result that is source-anchored yet, we stress, *reference-relative and individuation-sensitive*, and not a claim about which chain “is” Bitcoin. Two further results sharpen that caution into measurement rather than hedging. First, decomposing each chain’s *agreements* with v0.1.0 shows that **BSV is the only chain in the set that agrees with the reference on any axis it had previously diverged from and reverted** — 4 of its 11, against zero for every other chain: a displacement measure cannot distinguish agreement preserved from agreement re-created. Second, the three references **do not describe one ruleset**: the whitepaper and the November pre-release share no axis at all; the whitepaper and v0.1.0 share exactly one and **disagree on it**; and the pre-release and v0.1.0 share 3, agreeing on the proof-of-work function and differing on the other 2. The single consensus axis the whitepaper specifies is one the released client does not implement as described. So “the origin” is not one object. The contribution is the method and its reproducible engine; the numbers are its worked demonstration. **These artifacts carry no monetary value.**

1. Introduction

Bitcoin’s descendant chains — BTC, Bitcoin Cash (BCH), Bitcoin SV (BSV), eCash (XEC), Bitcoin Gold (BTG) — each present themselves, explicitly or implicitly, in relation to an origin. Comparisons of how far each has moved from that origin are common but usually *rhetorical*: a list of favoured changes, weighted by the author’s priorities, concluding with the author’s preferred chain. Such comparisons are unreproducible and unfalsifiable; a reader cannot re-derive them or locate where a disagreement lies. We take the opposite stance. Rather than argue *which* chain is closest, we build a **measurement instrument** that (i) makes the axes of comparison explicit and freezes them before each reported run, drawing them from documented change logs, (ii) records each chain’s value on each axis as a fact tied to a stated warrant — a primary source for all but the absence cells — and (iii) computes displacement from a reference by a stated rule, exposing coverage and sensitivity. The instrument does not decide the contested question of protocol identity; it makes one *bounded, well-posed* component of that question — consensus-rule displacement — reproducible. The design principle is that **reproducibility relocates inter-rater disagreement rather than removing it**. In content analysis and empirical software engineering, subjective codings are made credible by multiple independent coders reporting their agreement [Krippendorff, 2018, Easterbrook et al., 2008]. Here each specified cell is a consensus value determined by a stated criterion against a cited primary record — or, for a claim of absence, against chronology — and the scoring step is mechanical — so two coders working from the *same encoding* cannot disagree about the result. **They can still disagree about the encoding, and an earlier draft of this paper wrongly claimed they could not.** The shipped dataset contains the counterexample: v0.1.0’s block-size state is

labelled `no-dedicated-cap` and BSV’s `no-consensus-cap`, which name the same condition and are scored a *mismatch* because the match rule is string equality. **Section 5 measures what that costs.** The defensible claim is that the locus of judgement moves from scoring, where it is invisible, to individuation, where it is explicit and testable. This is the same move the surrounding project makes elsewhere — replacing narrative about the earliest Bitcoin with regenerable computation [Saxena, 2026a,b] — applied to cross-chain comparison.

2. Method

Profiles. A *profile* is a set of consensus-rule values. Three are **references**: the 2008 whitepaper [Nakamoto, 2008] (design intent); the 15 November 2008 pre-release [Satoshi Nakamoto Institute, 2026] (a partial, source-bounded snapshot); and v0.1.0 [Satoshi Nakamoto Institute, 2026] (the January 2009 client, the only complete early ruleset). Five are **chains** evaluated at a frozen date: BTC, BCH, BSV, XEC, BTG. All comparisons are reference→chain.

Chain selection. Which chains enter the comparison is itself a judgement, so the rule is stated before the results and does not depend on them. A chain is **eligible for inclusion if** it satisfies all three of: **(1) direct ledger ancestry** — its ledger continues Bitcoin’s genesis block, so no reissued or re-genesised chain qualifies however it is named; **(2) an active mainnet at the freeze** — producing blocks on 1 August 2026; and **(3) a public, dated primary record that explicitly identifies the relevant consensus divergences** — a BIP, an upgrade specification, or a self-describing commit. *Raw source diffs carrying no explicit statement of the consensus change fall outside this study’s audit boundary, which is a scope decision and not a claim that they cannot be measured; the subsection below explains why.* An earlier draft described the population as “large” descendant chains without defining “large”; the word is removed, because size was not a criterion and using it would have made the population a popularity judgement. *(We say “eligible for inclusion if”, not “included iff”: the criteria decide which chains qualify, and the candidate audit below applies them outside the set – but a hand-built candidate list cannot certify that every qualifying descendant has been enumerated.)*

An inclusion rule only applied to the chains one already chose is a description, not a rule, so we applied it to candidates outside the set. **Doing so changed this study.**

Bitcoin Gold satisfies all three criteria: it duplicated Bitcoin’s ledger through block 491,406, it publishes dated hard-fork specifications, and it was producing blocks at the freeze — we queried a primary chain endpoint and block 958,305 carries header time 2026-08-01T21:01:04Z. **It is therefore measured here as a fifth chain rather than argued away.** The alternative was to add a fourth criterion, and **a criterion invented after seeing which chain it removes is not a criterion**; the three criteria were fixed before being applied to the

candidate audit, and keeping them fixed when they produced an inconvenient answer is the only thing that makes fixing them beforehand mean anything.

Including it had a consequence we did not anticipate and report because it is the more interesting finding: **Bitcoin Gold changed the proof-of-work function, and no axis existed for that.** The axis set had been shaped, invisibly, by the chains already chosen. Adding `pow_function` — and `coinbase_height` (BIP34), which a referee identified independently — takes the enumeration to 19 axes and changes every reported rate. **The ordering is unchanged.**

Table 1: The selection rule applied to candidates outside the set, with each verdict’s evidence. A candidate excluded for a reason we have not checked is one we cannot defend, so unverified criteria are marked as such rather than assumed.

candidate	(1) ledger ancestry	(2) active at freeze	(3) dated primary record	verdict
Litecoin (LTC)	NO: separate genesis block	yes	yes	Fails (1). A new genesis is a new ledger, however similar the code
Dogecoin (DOGE)	NO: separate genesis, and a Litecoin derivative	yes	yes	Fails (1), twice over
Bitcoin Private (BTCP)	NO: fork-merge with Zclassic, not a continuation	UNVERIFIED	partial	Fails (1): a merged ledger is not a continued one
Bitcoin Diamond (BCD)	yes: forked from the Bitcoin ledger	UNVERIFIED	sparse/undated	Fails (3): no public dated specification record we could locate, which is an admission about our search as much as about the chain
Bitcoin Satoshi Vision testnets / regtest	n/a	n/a	n/a	Out of scope: not independent mainnets

Criterion (1) does most of the excluding, and does it on substantive grounds: 3 of the 4 candidates fail it because their ledger does not continue Bitcoin’s genesis block. **Criterion (3) excludes exactly 1** — and that is worth saying plainly, because the paragraph that follows defends criterion (3) at length and would otherwise read as a defence of the rule that did the work. It is not; it is a defence of the rule that has barely been used, which is a different and weaker thing.

Criterion (3) nonetheless excludes for a methodological reason rather than a substantive one. **An earlier draft justified it too strongly**, saying that a chain whose consensus changes exist only as source diffs *“cannot have a single source-anchored cell and so cannot be measured by this instrument at all.”*

That was false, and the paper itself is the counter-example

Criterion (3) admits “a BIP, an upgrade specification, **or a self-describing commit**”, and the v0.1.0 reference profile is anchored to **source files**, not to any prose specification. **A dated, content-addressed commit can source-anchor a consensus value** — we do it throughout.

⇒ **So this is a scope decision, not a technical impossibility, and it is restated as one.** What criterion (3) actually requires is a record from which the *relevant consensus divergences can be identified without our reading the diff and deciding for ourselves what changed*. A specification says which rule moved and when; a commit history obliges the analyst to infer it, and **that inference is exactly the unsourced judgement this method exists to remove.**

The cost is stated rather than hidden: the boundary is drawn where auditability becomes expensive, not where measurement becomes impossible, and a different study could legitimately draw it further out. A reader who supplies the missing record — or who does the diff-reading and cites it — can add a column, and the engine accepts it. **The contradiction was found by an external referee, who noticed that criterion (3) contradicted our own source model.**

A note on that label. The archive distributed as `bitcoin-0.1.0.rar` contains **v0.1.1, not the 8 January 2009 release** — its size matches the figure Satoshi states for `bitcoin-0.1.1.rar` in a 10 January 2009 message, and the shipped executable’s PE `TimeStamp` is 2009-01-10, two days after v0.1.0 was announced. Prior archival analysis reached this first [Stefanov, 2020]. **It does not affect anything reported here:** the v0.1.0-to-v0.1.1 delta is confined to `irc.cpp` and `serialize.h`, neither of which carries a consensus rule, so no axis or behaviour below changes. We retain the conventional filename because the published digests are recorded under it.

Axis selection — two classes. An axis is a *consensus-relevant* rule: one that can render a block or transaction valid on one profile and invalid on another. *Implementation-only* differences (for example, the ECDSA library) are excluded: we compare consensus behaviour, not code lineage. Axes are admitted under one of two stated classes, enumerated from documented change logs — the Bitcoin BIPs and release history, and the published Bitcoin Cash [Bitcoin Cash Node, 2026, bitjson and 0353F40E, 2023], Bitcoin SV [Bitcoin SV specification contributors, 2020, Bitcoin SV, 2026], eCash [Bitcoin ABC, 2026, eCash, 2024] and Bitcoin Gold [Bitcoin Gold, 2026] specifications, together with the relevant Bitcoin Improvement Proposals [Andresen, 2012a,b, Todd, 2014, Wuille, 2015, BtcDrak et al., 2015, Lombrozo et al., 2015, Wuille et al., 2020a,b] — the axes for P2SH, strict-DER signature encoding and Schnorr are anchored to BIP 16, BIP 66 and BIP 340 respectively, and the paper discusses all three:

(i) **Changed on a descendant** — the rule changed on at least one included chain since January 2009. **17 axes.**

(ii) **Reference-discriminating** — an early *reference* specifies the rule, and no descendant has changed it. **2 axes: the initial block subsidy and the target block spacing.**

The second class is a revision, reported as one

An earlier draft stated only class (i). **That rule contradicted its own dataset** — the subsidy, spacing and supply axes changed on no included chain — and an external referee found the contradiction. Enforced literally, those three axes leave the dataset.

We retained the subsidy and spacing axes under class (ii) — and dropped the supply axis, for the reason given below. The reason for retaining the two is not convenience: how the choice of reference changes what can be said is the paper’s subject, and deleting them would be cutting the instrument to fit one sentence of the rule.

THE ORIGINAL JUSTIFICATION FOR THIS CLASS NO LONGER HOLDS, AND IS REPLACED RATHER THAN REPAIRED *Status: the argument below is superseded and kept for the record; the current claim follows the arrow.*

This passage used to argue that without class (ii) the November 2008 reference would be left with **zero jointly specified axes and no result at all**, and that the subsidy and spacing were **the only axes that profile specifies**. Both statements were true when written and **both were falsified by our own later revision**: adding `pow_function` for Bitcoin Gold created a *class-(i)* axis that the November pre-release also specifies. Enforcing class (i) alone would now leave that reference with **one** jointly specified axis, not none.

⇒ **The correct claim is weaker and we state the weaker one: class (ii) buys additional reference coverage, not chain discrimination.** It raises November’s coverage from 1 axis to 3; it cannot separate the chains, because every chain agrees on both.

The dependency ran the wrong way round and

that is the lesson. A justification for a *design* rule was resting on a *dataset* fact, so extending the dataset silently invalidated the rationale while every number stayed correct. Found by an external referee in round 7, three rounds after the axis that broke it was added.

⇒ The class-(ii) revision is stated before the results so that its effect on every reported number is visible rather than absorbed. **We do not claim it was fixed before the analysis began — this section records that it was not, and why.**

The cost of that revision, stated here rather than discovered later

5 axes carry a single value across all five chains and therefore cannot separate them at all. 2 of those agree with v0.1.0 (initial block subsidy, target block spacing) and 3 differ from it (signature encoding, output-value range check, coinbase height commitment), so every chain receives the same fixed 2 matches and the same fixed 3 mismatches before any comparison begins. **Every mismatch rate reported under the v0.1.0 reference contains that constant component,** and only the remaining **14 of 19** do any comparative work — $5 + 14 = 19$, which a reader should be able to check on the page rather than take on trust. A fact we return to in Section 5 and which should be carried through every number below.

A third axis was removed because the rule, once stated, excluded it

An earlier draft carried an eighteenth axis, the **monetary supply schedule** (210,000-block halving, 21-million cap). It qualifies under neither class: no descendant has changed it, and no early reference specifies it. **The engine’s own class validator caught this within a minute of the rule being written.** Its presence had been an unstated judgement that the 21-million cap is too famous to omit — and there is no principled reason it was included while, say, BIP34’s coinbase-height rule was not. **The finding survives without the axis and is stated here instead: the halving interval and asymptotic cap are identical on v0.1.0 and on all five descendants.** An axis on which every profile agrees carries no comparative information.

The engine validates that the declared classes are **consistent with the frozen states** — a class-(ii) axis must be one on which every profile agrees *and* which an

early reference specifies — **and the validator fails if a cell edit breaks that.** What it cannot check is the historical *unchanged* condition, which is a separately sourced assertion: **the single-state schema cannot derive it**, for the same reason Section 4.1 gives — a chain that changed a rule and later restored it is indistinguishable, at the freeze, from one that did not touch it. The enumeration yields 19 axes (Table 2), spanning best-chain selection, the block-size rule, the script opcode vocabulary and its numeric/element limits, signature encoding and scheme, the output-value range check, P2SH, segwit, Taproot, timelock opcodes, the difficulty algorithm, replay protection, transaction ordering, and the early monetary parameters.

The axis list is admissible, not exhaustive

The two classes state **when an axis may qualify**. They do **not generate the set**, and this revision is the demonstration: a referee identified BIP34’s coinbase-height rule as an obvious class-(i) axis we had simply not listed, and testing our own chain-selection rule surfaced a proof-of-work axis that existed only because every chain we had chosen shared one value. **Two axes, added under two different external checks rather than by our own enumeration.**

Therefore we claim only that these are **19 explicitly specified macro-axes in the frozen instrument, drawn from documented consensus changes, admissible under the stated classes and not asserted complete. Not *pre-registered*:** two of them were added under external review after earlier results existed, as this section states in full. *The instrument is frozen before each reported run; it was not frozen before the project began, and claiming otherwise would contradict our own revision history.* Omitted-axis choice remains an unbounded source of model uncertainty, and the honest bound on it is that two omissions were found in one round. A future revision that wanted completeness would need a sourced consensus-change event ledger mapped onto axes, which this schema does not have.

One cell was reversed after a referee falsified a claim that depended on it, and that is disclosed here rather than absorbed

A previous revision encoded the **whitepaper’s proof-of-work function as sha256d, sourced to the whitepaper, at high confidence.** A referee falsified two claims resting on it. Re-reading the source, the encoding was wrong on two independent grounds: the

paper says the work is “*scanning for a value that when hashed, **such as with SHA-256**, ...*” — illustrative, not normative — and that it “*can be verified by executing **a single hash***”, which is not double-SHA-256 at all. **The cell is now unspecified.**

What that edit moved: whitepaper coverage from 2/19 to 1/19; the whitepaper and the November pre-release returned to sharing no axis; and two chains’ whitepaper rates moved — Bitcoin Gold’s from 0.50 to 0.00, because the proof-of-work mismatch left the comparison, and eCash’s from 0.50 to 1.00, because its one remaining jointly specified cell is best-chain selection, which the encoding labels **most-work+avalanche** for its avalanche post-consensus and which therefore mismatches the whitepaper’s **most-work** (Table 2 shows the label). That row therefore turns on a labelling decision: the label re-scoring of Section 5.1 is defined under v0.1.0 only, and a relabelling of that one cell under the whitepaper would move eCash’s row from 1.00 to 0.00 — a counterfactual stated by inspection of the one jointly specified cell, not computed by the engine. It is stated here so that the whitepaper row is not read as a finding about eCash. **It also restored a sentence the same referee had falsified**, which is exactly why it is flagged rather than quietly banked.

This section already holds that “a criterion invented after seeing which chain it removes is not a criterion.” A cell reversed after seeing which claim it falsifies is the same shape, and it earns the same treatment: state what it said, why it changed, and what it moved. **We think the new value is right. The reader is entitled to check that judgement against the old one rather than only against the corrected result.**

Cell schema. Each cell is {value, criterion, source, confidence}, where *source* is one or more primary records. Cells do **not** carry an activation height or date, which is why the instrument evaluates at a single frozen date and refuses any other (Section 7). The *value* is a canonical label; two profiles *match* on an axis iff their labels are equal. The *criterion* is the objective question the value answers (“is P2SH a consensus rule?”). The *source* is a primary record a reader can check. *Confidence* is **high** (unambiguous primary source) or **med** (documented but nuanced, and flagged). **Of the 152 cells, 34 are unspecified (None); of the 118 that carry a value, 111 are high-confidence and 7 are medium**, and the medium cells do not affect the ordering (Section 5). *An earlier draft reported “117 of 152 high-confidence” — a typed figure that no partition of the present cells reproduces, and one that mixed unspecified cells into the denominator; every count is now interpolated from the engine.* **None** marks that a profile does not specify an axis — kept distinct from “specified and equal.”

Mismatch rate and coverage. For a reference→chain pair, over the axes both specify (*jointly specified*): the **mismatch rate** is differing / jointly-specified, and is *undefined* where nothing is jointly specified; **coverage** is jointly-specified

/ total. We report both. A single “distance” number is rejected precisely because the three references have very different coverage, and a mismatch rate read without its coverage misleads.

Sensitivity. Because any such comparison depends on which axes are chosen and how finely they are individuated, the engine computes, for every reference→chain pair, four perturbations: the leave-one-axis-out range of the mismatch rate; a *merged-cluster* variant that collapses the post-2017 witness/signature upgrades (segwit, Taproot, Schnorr) into a single axis; the range over *every* subset obtained by dropping up to three of the 19 axes; and a re-scoring under alternative labels. Section 5 reports them for the v0.1.0 reference only: the three axis perturbations carry no information over one or 3 jointly specified axes, and the label re-scoring is defined under v0.1.0 (Section 5.1; the one labelling decision that moves a low-coverage row is stated in the proof-of-work box above); the other pairs’ axis perturbations are in `comparison.json`. We report a conclusion only to the extent it survives all four perturbations.

Evaluation date. All chain values are asserted as of the evidence freeze, **1 August 2026**, and the engine refuses to evaluate any other date (Section 7). Table 2 gives the axes and the value each profile takes; the machine-readable artifact records the criterion and the cited source for every specified cell. **The two earlier references are omitted from its body because they are almost entirely empty** — the whitepaper specifies one of these axes, the November 2008 pre-release 3 — and mostly blank cells would misrepresent them as sparse data rather than as documents that do not legislate consensus rules. Their values follow the table.

Table 2: The 19 consensus axes, with the v0.1.0 reference and the five chains.

#	Axis	v0.1.0	BTC	BCH	BSV	XEC	BTG
1	Best-chain selection	height	most-work	most-work	most-work	most-work+avalanche	most-work
2	Block-size	no-dedicated-cap	1mb+weight	abla-dynamic	no-consensus-cap	32mb	1mb+weight
3	Script opcode vocabulary	broad	restricted	partial-restore	broad	partial-restore	restricted
4	Script-number operand width	unbounded-openssl	4-byte	large-bigint	32mb-limit	8-byte	4-byte
5	Script element-size limit	none	520-byte	10000-byte	none	520-byte	520-byte
6	Signature encoding	lenient-openssl	strict-der	strict-der	strict-der	strict-der	strict-der
7	Output-value range check	none	moneyrange	moneyrange	moneyrange	moneyrange	moneyrange
8	Signature scheme	ecdsa-only	ecdsa+schnorr	ecdsa+schnorr	ecdsa-only	ecdsa+schnorr	ecdsa-only
9	Pay-to-Script-Hash (P2SH)	none	p2sh	p2sh	none	p2sh	p2sh
10	Segregated witness	none	segwit	none	none	none	segwit
11	Taproot output type	none	taproot	none	none	none	none
12	Timelock opcodes	nops	cltv+csv	cltv+csv	nops	cltv+csv	cltv+csv
13	Difficulty-adjustment algorithm	2016-block-retarget	2016-block-retarget	asert	daa-cw144	asert+rtt	lwma
14	Replay protection	none	none	forkid	forkid	forkid	forkid
15	Transaction ordering in a block	topological	topological	ctor	topological	ctor	topological
16	Initial block subsidy	50	50	50	50	50	50
17	Target block spacing	10-min	10-min	10-min	10-min	10-min	10-min
18	Coinbase height commitment	not-required	required	required	required	required	required
19	Proof-of-work function	sha256d	sha256d	sha256d	sha256d	sha256d	equihash-btg

The two earlier references, in full. The **2008 whitepaper** specifies axis 1 only, as *most-work*. The **15 November 2008 pre-release** specifies 3: axis 16 as *100*, axis 17 as *15 min*, and axis 19 as *SHA-256d*, and nothing else. Axis 19 is the one cell on which that pre-release and v0.1.0 both speak **and agree**; on the other 2 they differ. Every other cell for both is *missing* — the profile does not address that rule — which is kept strictly distinct from “specified and equal”, and is what produces the low coverage reported for both in Table 3. A rate computed over one or 3 jointly specified rules is not comparable with one computed over all of them, which is why coverage is printed beside every rate rather than in a footnote.

How much of the table discriminates. 5 axes (`sig_encoding`, `value_range_check`, `subsidy_base`, `block_spacing`, `coinbase_height`) carry the *same* value on every chain in the set. 3 of those disagree with v0.1.0 and 2 agree with it, so they contribute a fixed offset of 3 mismatches and 2 matches to every chain and **cannot affect the ordering**. The comparison between chains is therefore carried by **14 of the 19 axes** — a fact we state rather than leave a reader to derive, since it bears directly on how much separation the reported rates express.

3. Reproducibility

The method is one program plus one renderer. `obl_metric.py` produces every numerical result and table; `figures/mismatch_heatmap.py` renders Figure 1 deterministically from that engine’s axis-matrix output. The engine holds the axis table (with every value, its criterion, its primary source, and its confidence) and the computation of mismatch rate, coverage, and sensitivity. Running it prints the `reference×chain` table and writes `comparison.json` (the full cell-level record, including per-cell sources), `comparison.csv` (the summary), and `axis_matrix.csv` (the raw axis values). The figure regenerates from the same engine. Recording a chain’s rule after a future upgrade would be a one-cell edit with a new source under a *new* evaluation freeze — the instrument evaluates at one frozen date and refuses any other (Section 7) — after which the entire table and figure recompute. **Once the encoding is fixed there is no manual scoring step: the artifact performs the derivation deterministically.** The encoding itself is a human judgement — axis individuation, label choice and source interpretation — which Section 5 measures rather than denies.

4. Results

Table 3: Mismatch rate and coverage for every reference→chain pair, evaluated 1 August 2026. The mismatch rate is *undefined* where coverage is zero; a rate is not to be read without the coverage beside it.

reference	BTC	BCH	BSV	XEC	BTG
whitepaper	0.0000 (cov 0.053)	0.0000 (cov 0.053)	0.0000 (cov 0.053)	1.0000 (cov 0.053)	0.0000 (cov 0.053)
nov08	0.6667 (cov 0.158)	0.6667 (cov 0.158)	0.6667 (cov 0.158)	0.6667 (cov 0.158)	1.0000 (cov 0.158)
v0.1.0	0.6842 (cov 1.000)	0.7368 (cov 1.000)	0.4211 (cov 1.000)	0.7368 (cov 1.000)	0.7368 (cov 1.000)

Under the whitepaper, coverage is $1/19 \approx 0.053$ — a single jointly-specified consensus axis (best-chain selection). We print the row in full **because it is the row most easily quoted out of context**: four of the five chains score 0.00 against the whitepaper, which sounds like a strong result and is not one. It means only that those four agree with the whitepaper on the one axis both specify; the fifth, eCash, scores 1.00 on that same single axis because of how its fork choice is labelled (the proof-of-work box in Section 2 states this). A ranking built on one axis is degenerate: the whitepaper simply does not constrain enough of the protocol to situate chains against it, which is itself a useful and often-overlooked finding. *Under the November 2008 pre-release*, coverage is $3/19 \approx 0.158$ (3 specified parameters — the 100-coin subsidy, the 15-minute spacing, and the proof-of-work function). **The subsidy and spacing differ from every descendant. The proof-of-work function agrees with four of the five — and differs on Bitcoin Gold, which replaced it** — which is exactly why the November rate is 0.6667 against BTC, BCH, BSV and XEC and still 1.00 against BTG. *A single axis separating one chain’s rate from four others is the clearest illustration in the paper of how much a low-coverage reference rests on.*

Under v0.1.0 (full coverage), the comparison is well-posed. On the 19 axes the mismatch rates are BSV 0.4211, BTC 0.6842, and BCH = XEC = BTG 0.7368. The axis-level structure is shown in Figure 1.

4.1 Decomposing agreement with v0.1.0: retention and restoration

A chain can agree with the reference for two very different reasons: it did not change the rule (**retention**), or it adopted a change and later removed it (**restoration**). A bare mismatch rate cannot tell these apart, and the distinction is one of the two parts of the caution this method must carry (Section 5.1 measures the other, the labelling). We therefore report it as a count.

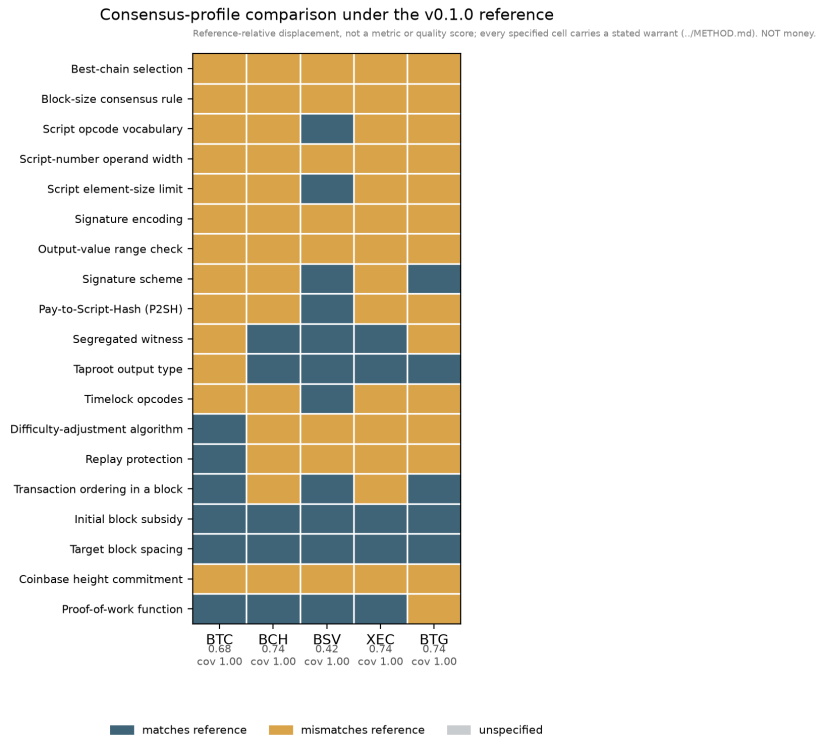


Figure 1: Axis-level agreement with the v0.1.0 reference. Each row is one of the 19 consensus axes and each column one chain. **Every cell is filled; the colour, not the fill, carries the meaning** — one shade marks a value differing from v0.1.0 and the other a value matching it. Regenerated by `figures/mismatch_heatmap.py` from the same engine that produces the tables.

Table 4: Decomposition of each chain’s agreement with v0.1.0. A match counts as a *restoration* only where the chain demonstrably **held a different value and later removed it**, with the introducing and removing change both named; every other match is a retention. Every chain’s matches include the 2 constant axes on which all profiles agree (initial block subsidy, target block spacing), counted as retentions.

chain	matches	retentions	restorations
BTC	6	6	0
BCH	5	5	0
BSV	11	7	4
XEC	5	5	0
BTG	5	5	0

BSV is the only chain in the set with any restorations at all, and it has four: the script opcode vocabulary and the element-size limit (both restricted by the August 2010 commit and re-enabled by the 2020 “Genesis” upgrade [Bitcoin SV specification contributors, 2020]); P2SH (a consensus rule on its lineage from 2012, removed by Genesis); and the **timelock opcodes** — CLTV and CSV reached its lineage through BIP65 and BIP112 in 2015–16 and Genesis sunsets them, the specification stating that the operations *revert to NOPs, which have no effect*, which is v0.1.0’s value. **Its other 7 agreements — including the absence of segwit, Taproot and CTOR — are retentions: BSV forked from Bitcoin Cash in November 2018 and has not held any of them.**

⇒ The caution is therefore more specific than a bare mismatch rate suggests: **4 of BSV’s 11 agreements are restorations: axes on which the chain diverged from v0.1.0 and later returned to the reference value.** A low mismatch rate can arise from removing later additions as readily as from preserving original ones, and it is not a verdict of authenticity, continuity, or intent.

A note on how this table was arrived at, because it bears on the method. An earlier version of this analysis inferred restoration from *which source a cell cites*, on the reasoning that a cell citing a chain’s own upgrade specification marks a rule that chain legislated. **That inference is false and the numbers it produced were wrong** — it reported BSV at 7 restorations and gave BCH and XEC two apiece. A citation records where a value is *documented*, not what the chain previously did: BCH forked three weeks before segwit activated and so did not have segwit to remove, yet its segwit cell cites the fork specification. **The table is now an explicit list in which every restoration names the change that introduced the rule and the change that removed it**, and nothing is inferred. *A citation is not a history.*

4.2 The references do not agree with each other

The engine compares references to chains. Asking it to compare the references *to each other* yields the result we consider most important in the paper:

Table 5: Reference-against-reference disagreement. Every pair of candidate origins, over the axes both specify. **They partly agree and partly disagree** — the whitepaper and the November pre-release share no axis at all; the November pre-release and v0.1.0 share 3 and differ on 2 — so the choice of reference is not a technical preliminary to the question but the substance of it.

pair	jointly specified	differing	axes
whitepaper vs nov08	0	0	(no overlap – undefined)
whitepaper vs v0.1.0	1	1	fork_choice
nov08 vs v0.1.0	3	2	subsidy_base, block_spacing

The single consensus axis the whitepaper specifies is one the released client does not implement as described. The paper describes best-chain selection by accumulated proof-of-work; the January 2009 client selects by height. **Of the 3 axes the November pre-release shares with v0.1.0, 2 differ and one agrees** — the subsidy and the block spacing changed between the preview and the release; the proof-of-work function did not.

⇒ **“The origin” is not one object.** Displacement is measured from whichever origin is chosen, and the available origins do not describe one ruleset: of the three pairs, one shares no axis at all; one shares a single axis and differs on it; one shares 3 and differs on 2. This is the strongest available argument for the paper’s own insistence that its output is *reference-relative*, and we would rather state it than have a reader discover it.

5. Robustness

Leave-one-axis-out. The v0.1.0 mismatch rates stay within BSV [0.3889, 0.4444], BTC [0.6667, 0.7222], and BCH, XEC and BTG [0.7222, 0.7778]; no single axis drives the ordering.

Dropping up to three axes — and why this is weaker than it looks. Over all 1160 subsets obtained by dropping up to three of the 19 axes the ranges are BSV [0.3125, 0.5000], BTC [0.6250, 0.8125], and BCH, XEC and BTG [0.6875, 0.8750], and the lowest-mismatch chain is BSV in 1160 of 1160. **But that is arithmetic, not a finding.** All five chains share the denominator 19, so the mismatch *counts* (8, 13, 14, 14, 14) order the rates, and the gap from BSV to the runner-up is 5. Dropping k axes moves any count by at most k , so the ordering *cannot* change for $k < 5$: invariance up to 4 dropped axes is a theorem. Exhaustively, the first ties appear at $k = 5$ (21 of 11628 subsets) and the first strict reversal at $k = 6$ (7 of 27132). **We report this because a referee found it, and because a robustness claim that is secretly a tautology is worse than none at all.**

Individuation. Collapsing segwit, Taproot and Schnorr into one axis moves every chain, and we report all five rather than the two that suit the argument:

Table 6: Merged individuation. The post-2017 witness and signature-scheme axes are collapsed into a single axis, and every rate recomputed. **Every chain moves, and they do not move in the same direction** — which is the point: individuation is not a neutral bookkeeping choice.

chain	base	merged	change
BTC	0.6842 (13/19)	0.6471	-0.0372
BCH	0.7368 (14/19)	0.8235	+0.0867
BSV	0.4211 (8/19)	0.4706	+0.0495
XEC	0.7368 (14/19)	0.8235	+0.0867
BTG	0.7368 (14/19)	0.8235	+0.0867

(Rates are given to four places, with the exact fraction beside the base rate. *0.625* and *0.8125* are exact binary values on a rounding tie, and two-place rounding of them is convention-dependent — an earlier draft printed *0.63*, which is not a number the engine emits under any rounding mode.) An earlier draft described this cluster as “BTC-only” and reported only BTC and BSV. That was wrong in both respects: Schnorr is a divergence for BCH and XEC as well, and **the merge moves those two chains further than it moves BTC**. The merge moves **every** chain and not in one direction: BTC -0.0372, BSV +0.0495, and BCH, XEC and BTG +0.0867 each — reported in full rather than for the two that suit the argument. BSV remains lowest under the merged individuation.

Confidence. The 7 medium-confidence cells (script-limit, signature-scheme and timelock details on BCH, BSV and XEC) lie inside all of these ranges and do not change the ordering. **Exhaustively, and now computed rather than asserted: confidence_sensitivity() enumerates all 128 assignments of those cells to match or mismatch, and BSV is uniquely lowest in 128 of them.** This result was previously stated in the text and produced by no code; it is emitted to `tables/figures.json` so that it follows a cell edit instead of outliving one.

5.1 Label granularity — the sensitivity of the encoding itself

Two profiles match iff their labels are **equal as strings**. The choice of label is therefore itself a coding decision, and the shipped dataset contains a case where two labels name the same condition:

```
v0.1.0  block_size_rule = "no-dedicated-cap"
BSV     block_size_rule = "no-consensus-cap"
```

Both mean *no consensus rule caps the block size*. **Scored as a mismatch purely because the strings differ.** The same applies to `unbounded-openssl` versus `unbounded` on script-number width. Conversely, a coder could reasonably argue the opposite way on `script_opcodes`: BSV’s post-Genesis vocabulary is not v0.1’s, since it adds `OP_CHECKDATASIG`, `OP_SPLIT` and `NUM2BIN/BIN2NUM` while still disabling `OP_2MUL`, `OP_2DIV`, `OP_VERIF` and `OP_VERNOTIF`.

Table 7: Label granularity. Each row re-scores the v0.1.0 comparison under one defensible alternative *labelling* of states that are already agreed as facts. **Only BSV moves.** Relabelling shifts it 0.0526 from its base rate against 0.0322 for removing any single axis — **one cell’s individuation can matter more than an entire axis** — though dropping up to three axes moves it further still, 0.1086.

relabelling	BTC	BCH	BSV	XEC	BTG
as published	0.6842	0.7368	0.4211	0.7368	0.7368
block-size	0.6842	0.7368	0.3684	0.7368	0.7368
labels unified					
script-number	0.6842	0.7368	0.4211	0.7368	0.7368
qualifier dropped					
(inert since the BSV cell became 32mb-limit)					
BSV opcode set	0.6842	0.7368	0.4737	0.7368	0.7368
individuated the two	0.6842	0.7368	0.3684	0.7368	0.7368
unification cases together					

⇒ **BSV SPANS [0.3684, 0.4737] UNDER RELABELLING**
— **WIDER THAN LEAVE-ONE-OUT [0.3889, 0.4444].**

None of these is a correction. Each is a labelling a competent coder could have chosen from the same primary sources. **The spread is the result**, and it is the honest answer to what an earlier draft of this paper claimed when it said there was “*nothing for two independent coders to disagree about*”. There is: they can disagree about individuation, and that disagreement moves the score 0.0526 — more than removing any single axis does (0.0322), though less than dropping up to three (0.1086).

This does not weaken the instrument; it locates its soft joint. A displacement measure defined by label equality inherits every judgement embedded in the labels. Reporting that explicitly is the difference between a measurement and an assertion. **The soft joint was identified in review, not in construction** — which is itself evidence about where a label-equality measure is weakest.

6. Interpretation

The output is **reference-relative consensus-rule displacement**, nothing more. It is *not a metric*, and the reason is stronger than an earlier draft stated. That draft said identity-of-indiscernibles fails because “*distinct profiles can share a mismatch rate*” — but sharing a value is not a failure of that axiom. **The**

real failure is that distinct profiles can sit at distance ZERO. Take two axes and profiles $A = (0, \emptyset)$, $B = (0, 1)$, $C = (1, 1)$. Under skip-unspecified $d(A, B) = 0$ although $A \neq B$; and $d(A, C) = 1$ while $d(A, B) + d(B, C) = 0 + \frac{1}{2}$, so the triangle inequality fails too. We therefore do not call it a metric. It is *not a quality score* and *not a ranking of which chain “is” Bitcoin*: protocol identity also involves naming, continuity, adoption, and governance, none of which this instrument measures or should. What the instrument does provide is a way to state — and have anyone re-derive — that, on a fixed, source-anchored set of consensus axes, chain X’s rules differ from reference Y on a specific fraction of the jointly-specified axes, with the sensitivity of that fraction made explicit.

7. Limitations

Granularity. The axis enumeration, though rule-governed, still embodies a choice of granularity; the four sensitivity analyses bound but do not eliminate its influence. Relatedly, only 14 of the 19 axes discriminate between chains at all (Section 2), so the reported rates contain a fixed component that carries no comparative information.

Coverage. Coverage is intrinsically low for the two earlier references, which limits what can be said against them — a limit we report rather than paper over, and which Section 4.2 shows to be more than a technicality: **the only axis on which any two references agree is the proof-of-work function, and they differ on every other axis they share.**

No historical evaluation. Values are asserted at a single frozen date, 1 August 2026. The engine accepts an `--at` argument and **refuses** any other date, earlier or later. It previously accepted one and returned byte-identical output, because no cell records when its rule activated; that silent no-op has been replaced by an explicit refusal naming what is missing. Making the instrument answer historically would require replacing each chain cell’s single value with a sourced timeline — roughly 95 cells, each needing the activation date its primary source states. That is real work, it is not done, and we prefer to say so than to ship a flag that appears to do it. **We note this because a paper whose claim is reproducibility is exactly the paper in which an inert control is most damaging: a reader who finds one has no way to judge which other capability is also decorative.**

A population that is eligible, not proven closed. The chain-selection criteria were fixed before application to the candidate audit, and that audit applies them to candidates outside the set — which is how Bitcoin Gold was found to qualify and became a fifth measured chain rather than a fourth criterion. **But passing that test on a hand-built candidate list does not prove the list is complete.** The criteria say which chains are *eligible*; they do not certify that every eligible descendant has been enumerated. We therefore claim only that **these five qualifying chains were identified by the candidate**

audit, and not that they are every chain the rule admits. This is the same posture as the axis set: **admissible, not exhaustive** — and both weakenings were forced by external checks rather than found internally, which is itself the honest thing to report about the method.

Scope. The method is deliberately confined to consensus rules; it says nothing about the economic, social, or governance dimensions of the chains it compares, and must not be read as if it did. It generalises beyond Bitcoin to any protocol lineage with documented change logs, which is its intended contribution.

Source attribution, and what a census of it found. The method’s premise is that a reader may contest any cell against its cited source, which requires the citation to be the *right* one. We therefore audited the citations mechanically rather than asserting them: `audit_btc.py` fetches the BIPs repository and confirms **8 of 8** BIP-backed BTC probes (over 7 cells; one cell is probed through two BIPs), and `audit_descendants.py` fetches the Bitcoin SV, Bitcoin Cash and Bitcoin ABC primary specifications and confirms **18 of 18** probes across BCH, BSV and XEC. Each fetch carries a control requiring the document to name itself, so a challenge or error page cannot be read as an answer. **The same script also probes Bitcoin Gold’s own chainparams.cpp for the 5 cells that distinguish it, and confirms 5 of 5**, so all 5 are recorded in `tables/audit_descendants.json` alongside the others. (*A third script, `audit_btg.py`, is not a citation audit at all: it tests chain-selection criterion (2) — whether Bitcoin Gold was producing blocks at the freeze — and it is why Bitcoin Gold is measured here rather than excluded. It emits no citation ledger because it verifies no cell.*) **Two units are in play here and they must not be added across.** The non-BTG probes number 26 over 24 distinct *cells* — a cell can be probed more than once: BTC’s timelock opcodes through BIP65 and BIP112, and BSV’s script-number width through two specifications. Adding Bitcoin Gold’s 5 probes, none of which repeats an earlier cell, gives **31 probes over 29 cells**. The fetched figure quoted throughout this paper is the *cell* count. **The split is by chain, not by script**, because Bitcoin Gold’s cells were the ones added last.

Three denominators appear in this paper and they are not interchangeable: **152** total profile cells, **118** of them carrying a value across all profiles, and **95** specified cells on the five *chains* — which is the denominator the audit partition uses, because that partition is defined over the descendant-chain cells; the three historical reference profiles are source-anchored separately and are not part of it.

Those are pass rates on the probes that were run, and a pass rate is not a coverage figure. The denominator matters more than the numerator, so the engine partitions all 95 specified cells by *what each one’s warrant actually is*:

Table 8: Audit coverage, partitioned by what each cell’s warrant actually is. **A pass rate on the probes that were run is not a coverage figure; this is the denominator.**

warrant	cells
fetched – a primary source retrieved and matched mechanically	29
inherited – argued from an ancestor pre-dating every fork in the set	51
absence – unconfirmable by construction	9
unclassified – anchored to a cited source, not yet fetched	6
total specified	95

The unclassified group is the honest residue and it is not empty; several of its cells are also the medium-confidence ones, which is why the confidence sensitivity in Section 5 matters. **We state this partition rather than the two pass rates alone, because the pass rates read as a census and are not one.**

The census changed two citations and **no values**. BCH’s script-integer cell cited the May 2022 upgrade, which delivered 64-bit integers; arbitrary precision arrived with CHIP-2024-07 BigInt, activated 15 May 2025 — still before the freeze, so the value stood and only the source was wrong. BSV’s signature-scheme cell cited the Genesis specification, which contains no occurrence of *ECDSA* or *Schnorr*; it is an absence, and is now cited and scored as one. Separately, three attributions warrant a reader’s attention without being errors: strict-DER encoding on BCH, BSV and XEC is inherited from BIP66 (2015) rather than originating at the 2017 fork, and BSV’s CW-144 difficulty algorithm originates in the November 2017 Bitcoin Cash upgrade rather than the August 2017 fork.

Claims of absence are the dataset’s weakest footing, and they are structurally so. The absence-warranted cells — 9 of them: **segwit on BCH, BSV and XEC** (Bitcoin Gold forked after segwit activated and therefore has it), **Taproot on BCH, BSV, XEC and BTG**, and **the Schnorr signature scheme on BSV and BTG** — cannot be confirmed by a fetch, because **no document establishes that a rule was not adopted** and so no fetch can return a document asserting the value: a specification can record a removal, as Bitcoin SV’s Genesis specification does for P2SH and the element-size limit, but non-adoption leaves nothing to cite. They rest on chronology, a fork cannot remove what it did not have, together with the absence of any upgrade specification introducing them. This is weaker than a positive citation, it is not remediable by searching for a record that non-adoption does not produce, and it is reported here rather than left for a reader to infer from the fact that the audit did not cover it. *The partition is by warrant, not by label. Of the 9, seven carry the value **none** and the two Schnorr cells carry **ecdsa-only**; three other chain cells carry **none** — BTC’s replay protection, inherited from v0.1.0, and BSV’s P2SH and element-size limit, documented removals — and are classified as inherited or fetched accordingly, so Table 2 shows ten **none** cells on the chains (a count by inspection of Table 2, not an engine figure) and the partition shows 9 absence cells.*

8. Conclusion

Situating chains against a historical reference need not be advocacy. By making the axes explicit and freezing them before each reported run, anchoring every value to a stated warrant — a primary source for all but the absence cells — reporting coverage alongside mismatch, and exposing sensitivity, the comparison becomes a computation a reader can rerun and contest cell by cell — reproducibility standing in for the inter-rater machinery a subjective coding would need. The worked result is itself instructive: the “distance from the origin” question is degenerate against the whitepaper, low-coverage against the preview, and, against v0.1.0, yields an ordering that is source-anchored yet reference-relative and individuation-sensitive — which is precisely why it should be read as a measurement, not a verdict. Two of the instrument’s own outputs make that case better than the argument does. The chain with the fewest mismatches is also the only one that reaches *any* of its agreement by removal rather than preservation — 4 axes of 11, with the other 7 recorded as retentions — and a displacement measure is constitutionally unable to tell the two apart; and the three candidate origins do not describe one ruleset — of the three pairs, one shares no axis at all; one shares a single axis and differs on it; one shares 3 and differs on 2 — so the choice of reference is not a technical preliminary but the substance of the question. **A measurement that discloses what it cannot see is more useful than one that does not, and building the disclosure into the engine keeps it in every rerun rather than in a footnote.**

Data and Code Availability

Every numerical result and table in this paper regenerates from `obl_metric.py`, and `figures/mismatch_heatmap.py` renders Figure 1 deterministically from the engine’s axis-matrix output. The engine carries the axis dataset it embeds. Running it writes `comparison.json` (the full cell-level record, including the criterion, cited source and confidence for each of the 152 cells, 34 of which are unspecified), `comparison.csv` (the summary), and `axis_matrix.csv` (the raw axis values); the figure regenerates from the same engine. **The replication package is archived on Zenodo under the concept DOI <https://doi.org/10.5281/zenodo.21964446>, which resolves to the latest version — 1.0.6 for this text, whose own version DOI is printed in that record; version 1.0.0, deposited on 16 August 2026, remains at <https://doi.org/10.5281/zenodo.21964447> — and the same artifacts are in the repository at <https://github.com/original-bitcoin-laboratory/genesis>, under `paper-artifacts/obl-metric/` at the signed tag `obl-metric-v1.0.6`. *The DOI is the durable address; the signed tag identifies the frozen repository version, and the commit it resolves to is recorded in the Zenodo version record (a document cannot carry the digest of the commit that contains it); the repository path is a convenience that may move.* **Every rate, coverage figure and engine count reported in this paper is interpolated from the engine’s output****

when the manuscript is built, not typed — the two statements made by inspection rather than by the engine (the eCash counterfactual in Section 2 and the `none`-label count in Section 7) say so where they stand — so the reported values are re-derived on every build rather than compared against the engine as of a date. The digests below identify the engine outputs from which the reported results can be independently reconstructed.

The bytes, not just the address. A repository URL is a promise: it can be edited after publication and a reader cannot tell. So the scoring, audit and figure artifacts underlying every reported result name themselves, by full SHA-256:

```
obl_metric.py
c57741458a459169e909aefdf5c1516e0460458c48d7c80d311770edd93752c2
audit_descendants.py
74daf47d77aa8266561963f8e4115eaa7376859450c1c9b923b8f94f40240214
audit_btc.py
daa7dcaebc464a206881be1107b463b510a61a86906d747012e7bc2013e01369
audit_btg.py
8de0a38b4e968662f2ea0e2d604a1c77a55ca2874bc54e5a381b2165679a5f65
figures/mismatch_heatmap.py
e5b3c633ef75ddd8171a8bb3036eb7201cddccfe883c8ee34cc4509fe6af71f6
figures/mismatch_heatmap_v010.png
d6ace5be32c6f94c7d78f36817bf93f0fdeed5af9e5ebee2af4af5b2f996b9de
```

```
tables/audit_descendants.json    run 2026-08-14T22:39:05+00:00
a35b7def457c9bda17d8c05edec334fb2aec827858cd6706c42569727c184ebc
tables/audit_btc.json          run 2026-08-14T22:38:41+00:00
ab63eef2f14674fcce02149a8d4d9136164706e551722bd535b2428d786f1efb
```

```
artifacts/comparison.json
2209d84e4cf03c7297016fec4cda05d23b00331ce3bc42601c5729801528bcc9
artifacts/axis_matrix.csv
191d70b5ee1206ec42e9184fbb5ce2624c99bd94788087b1f87a5c1f0175aaf2
artifacts/comparison.csv
71bb2fa5a06cf72a68ef6fa80578fc1eab3ed0d04da432518c675b46eb254e18
```

```
tables/table*.md    manifest over the 8 engine-emitted table files
11afe004a45e10c0fd655aff7767d47a41e6f02e1158fba3c55028a751a8f221
```

Any copy that does not hash to these is not the copy this paper reports on. **The two `tables/audit_*.json` entries are the audit ledgers, and they are what tie the reported audit figures to recorded output rather than to source files that merely existed.** Each records its own run timestamp and, per probe, the URL fetched, the HTTP status, the control outcome and the SHA-256 of the retrieved body; the 31 probe records over 29 distinct cells reported in Section 7 are the contents of those two files — Bitcoin Gold’s 5 included, since `audit_descendants.py` performs them.

A digest change is therefore legible, within what a digest can establish. If a ledger digest changes, the recorded audit output changed, and the embedded run timestamp identifies the execution that the ledger records. If only a script digest changes while its ledger digest and run timestamp do not, the published audit output has not been regenerated under those new script bytes. A digest proves which bytes exist, not how they came to exist: the timestamp is a recorded claim about an execution, not proof that one occurred. The narrower statement is the one the evidence supports.

The three artifacts/ entries are the engine’s principal serialised outputs, and they give the engine the counterpart the ledgers give the audit scripts. `comparison.json` pins the full cell-level comparison record, `axis_matrix.csv` pins the canonical axis matrix, and `comparison.csv` pins the summary rates. Every numerical table in this paper is emitted by the same engine run whose comparison state those files serialise.

Their digests establish byte identity, not the semantic reason for a change. A changed digest means that artifact changed; a diff identifies whether the change touched values, sources, confidence, criteria or serialisation. *comparison.json records criteria, primary sources and confidence alongside values, so a citation-only correction moves its digest while every number in this paper stays the same — this work has made exactly such corrections.* Conversely, if the engine digest changes while these three remain byte-identical, what is established is that the serialised comparison state and summary outputs are unchanged.

The final entry pins the remaining link between that state and a printed table. The manuscript’s numerical tables are substituted from files the engine emits, and the manifest is one SHA-256, computed by `obl_metric.py` over each emitted table’s file name (as UTF-8 bytes) followed by the raw 32-byte SHA-256 of its contents, with no separator, in sorted file-name order, so those 8 files are pinned without adding 8 lines. *One presentation artifact remains deliberately outside the list: tables/figures.json, which carries the scalar substitutions — it is the file every digest above is recorded in, so it cannot contain its own. revision_check_live.py covers it the other way round, by re-deriving each scalar the paper prints from the engine and failing on any difference.*

A generation timestamp was deliberately not embedded, and the distinction is the point: it would change the digest on every run and convert “the output changed” into “time passed”, destroying the property the pairing exists to provide. The audit ledgers carry timestamps because they record network fetches, which are not reproducible — the descendant ledger’s run is stamped 2026-08-14T22:39:05+00:00, after the 1 August freeze it audits; these record computations, which are. The writers disable platform line-ending translation, so these digests are properties of the data rather than of the machine that produced them. The digests cover those files only — not the fetched upstream sources, whose own hashes are recorded per-probe inside the ledgers, and not tables/figures.json, which cannot be listed here because it is the file these

digests are recorded in. Recording any chain’s rule after a future upgrade would be a one-cell edit carrying a new primary source under a new evaluation freeze, after which the entire table, the sensitivity analysis and the figure recompute for that date; the 1 August 2026 results are not edited in place. **Once the encoding is fixed there is no manual scoring step: the artifact performs the derivation deterministically**, and a reader who disagrees with a value can contest it cell by cell against the cited source. *That is not a claim that no human judgement is present — it is present in the encoding, and Section 5.1 measures how much the result moves when a defensible alternative labelling is substituted.*

Scope statement. The experimental chains and artifacts associated with this work carry no monetary value: there is no premine, no sale, no token, and no price. Nothing in this paper is financial advice, and the instrument takes no position on whether any chain should change anything. **The instrument compares five chains that do carry a market price; it measures none of it, and a mismatch rate must not be read as a statement about any of them as an asset.**

References

- Gavin Andresen. BIP 16: Pay to script hash. <https://github.com/bitcoin/bips/blob/master/bip-0016.mediawiki>, 2012a. Bitcoin Improvement Proposal; the P2SH output type, activated on Bitcoin mainnet 1 April 2012. Primary source for the P2SH axis.
- Gavin Andresen. BIP 34: Block v2, height in coinbase. <https://github.com/bitcoin/bips/blob/master/bip-0034.mediawiki>, 2012b. Bitcoin Improvement Proposal; activated on Bitcoin mainnet at height 227,931.
- Bitcoin ABC. Bitcoin ABC source: MAX_SCRIPT_ELEMENT_SIZE and MAX_SCRIPTNUM_BYTE_SIZE. <https://github.com/Bitcoin-ABC/bitcoin-abc>, 2026. For a chain the implementation is the consensus rule.
- Bitcoin Cash Node. Bitcoin Cash upgrade specifications. <https://upgradespecs.bitcoincashnode.org/>, 2026. May 2018 re-enabled opcodes; Nov 2018 CTOR; May 2019 Schnorr; Nov 2020 ASERT; May 2024 ABLA; May 2025 VM Limits and BigInt.
- Bitcoin Gold. BTCGPU consensus parameters: Equihash-BTG and LWMA. <https://github.com/BTCGPU/BTCGPU>, 2026. `src/chainparams.cpp`: BIP34Height = 227931, SegwitHeight = 481824, nPowTargetSpacing = 10*60; ledger duplicated through block 491,406.
- Bitcoin SV. Chronicle upgrade specification. <https://github.com/bitcoin-sv/specs/protocol>, 2026. Script-number consensus limit raised 750 KB to 32 MB; mainnet activation height stated as 943,835 in the specification and 943,816 in the v1.2.0 release notes.

- Bitcoin SV specification contributors. Genesis upgrade specification, 2020. URL <https://github.com/bitcoin-sv-specs/protocol/blob/master/updates/genesis-spec.md>. bitcoin-sv-specs/protocol, updates/genesis-spec.md; activated 4 February 2020; removes the block-size consensus cap and P2SH, restores opcodes and makes limits miner-configurable; accessed August 2026.
- bitjson and 0353F40E. CHIP-2023-04: Adaptive blocksize limit algorithm for Bitcoin Cash. <https://gitlab.com/0353F40E/ebaa>, 2023. Activated on Bitcoin Cash mainnet 15 May 2024.
- BtcDrak, Mark Friedenbach, and Eric Lombrozo. BIP 112: CHECKSEQUENCEVERIFY. <https://github.com/bitcoin/bips/blob/master/bip-0112.mediawiki>, 2015.
- Steve Easterbrook, Janice Singer, Margaret-Anne Storey, and Daniela Damian. Selecting empirical methods for software engineering research. In *Guide to Advanced Empirical Software Engineering*, pages 285–311. Springer, 2008. doi: 10.1007/978-1-84800-044-5_11.
- eCash. Real time targeting (heartbeat). <https://e.cash/>, 2024. A second difficulty applied on top of the base ASERT DAA; blocks exceeding the real-time target are rejected by the Avalanche layer.
- Klaus Krippendorff. *Content Analysis: An Introduction to Its Methodology*. SAGE Publications, 4th edition, 2018.
- Eric Lombrozo, Johnson Lau, and Pieter Wuille. BIP 141: Segregated witness (consensus layer). <https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki>, 2015.
- Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008. URL <https://bitcoin.org/bitcoin.pdf>.
- Satoshi Nakamoto Institute. Code: the earliest Bitcoin source archives, 2026. URL <https://satoshi.nakamotoinstitute.org/code/>. The 15 November 2008 preview and the archive published as v0.1.0; accessed August 2026. The latter is sha256 8b17eb9a5707f2519defda4cdf8d14fa1b8dee630e11e6ef85ff9f5547555b56, identified by internal evidence as v0.1.1, built 10 January 2009. Its consensus rules are identical to v0.1.0’s, so no result here is affected.
- Parth Mauria Saxena. Bitcoin’s earliest source codebases: A hash-matched, reproducible differential study of the November 2008 preview, the SNI-labelled v0.1.0 archive, and v0.1.3. <https://doi.org/10.5281/zenodo.21513374>, 2026a. Preprint and replication package, archived on Zenodo.
- Parth Mauria Saxena. Executing the earliest Bitcoin: A hash-anchored reconstruction of the January 2009 client and a source-bounded model of the November 2008 preview, 2026b. Manuscript under review. Its historical-binary evidence (two-node runs of the unmodified January 2009 client) is archived on Zenodo at <https://doi.org/10.5281/zenodo.21978132>.

- G. Stefanov. Satoshi used timestamps for early Bitcoin source code versioning, November 2020. URL <https://chainbulletin.com/satoshi-used-timestamps-for-early-bitcoin-source-code-versioning>. The Chain Bulletin, 19 November 2020. Archival timestamp-and-version analysis arguing that the surviving `bitcoin-0.1.0` archive is the later v0.1.1 revision.
- Peter Todd. BIP 65: OP_CHECKLOCKTIMEVERIFY. <https://github.com/bitcoin/bips/blob/master/bip-0065.mediawiki>, 2014.
- Pieter Wuille. BIP 66: Strict DER signatures. <https://github.com/bitcoin/bips/blob/master/bip-0066.mediawiki>, 2015. Bitcoin Improvement Proposal; primary source for the strict-DER value on the signature-encoding axis, and for the inheritance of that value by every descendant that forked after activation.
- Pieter Wuille, Jonas Nick, and Tim Ruffing. BIP 340: Schnorr signatures for secp256k1. <https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki>, 2020a. Bitcoin Improvement Proposal; primary source for the ecdsa+schnorr value on the signature-scheme axis.
- Pieter Wuille, Jonas Nick, and Anthony Towns. BIP 341: Taproot: SegWit version 1 spending rules. <https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki>, 2020b.